

LogProof

Platform Working Guide

STATUS	OWNER	LAST UPDATED
Prototype guide	LogProof project	September 25, 2026

Authors	Project team (not assigned)
Reviewers	Not assigned
Related docs	README.md; WitFoo dataset card (link in §6)
Scope	Current demo behavior, controls, data path, guarantees, and production gaps.

1. Abstract

LogProof is a local-first log preprocessing prototype. It accepts varied security and operations events, stores original bytes, parses them into a shared event record, and keeps field-level links for review. Operators can inspect evidence, detect shape changes, compare a candidate firewall parser with a fixed corpus, and approve or roll back that example parser. The current build demonstrates a review workflow; it does not claim production scale, source authenticity, or SIEM integration.

The demo supports individual events and small batches on a single local service. The API accepts up to 256,000 bytes per event; NDJSON batches allow up to 100 records, 2 MB total, and 256,000 bytes per event. SQLite stores metadata and local files store raw bytes. Authentication, high availability, distributed throughput, and ministry deployment are future engineering work.

2. Goals and Non-Goals

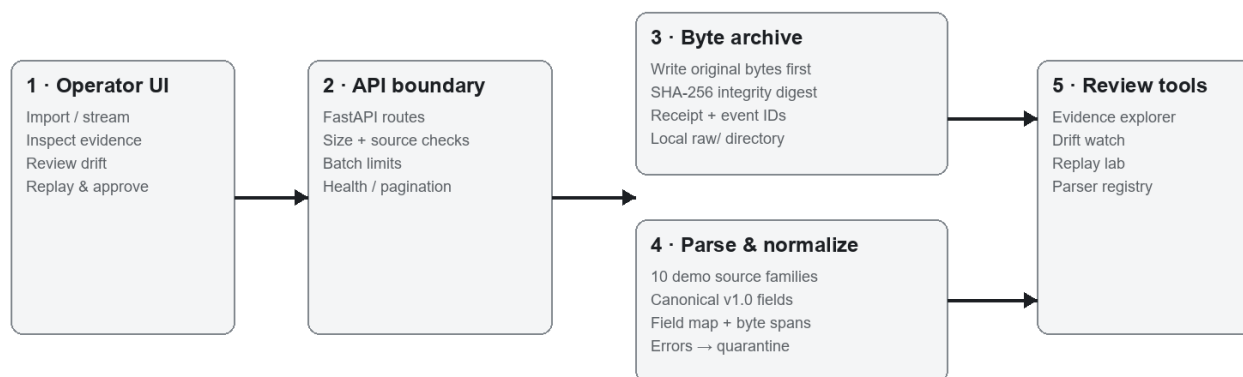
Goals	Non-goals
Retain and inspect received bytes.	Prove source authenticity or trusted chain of custody.
Normalize varied demo formats with traceable field maps.	Claim full OCSF coverage or universal parser support.
Demonstrate drift detection and gated firewall replay.	Provide production HA, multi-tenant auth, or billion-event scale.
Run locally or in the included container setup.	Deliver ministry-ready deployment or SIEM export today.

3. Background and Problem Statement

Security logs arrive as JSON, Syslog, CEF, web access lines, XML, CSV, and vendor-shaped records. Source-specific parsing makes investigations harder when the original bytes, parser version, and field interpretation cannot be reviewed together. LogProof puts a traceable preprocessing step before a SIEM or data lake. The prototype demonstrates ten source families across those formats; it preserves bytes and maps parsed fields but does not yet normalize to full OCSF or connect to a SIEM.

The browser presents ingestion and review tools. A FastAPI service validates source and size, saves original bytes, then calls a source-specific parser. SQLite holds event, receipt, field-map, quality, drift, and parser-registry metadata. The invariant is simple: a normalized value must remain traceable to the received bytes and parser version. SHA-256 verifies stored-byte integrity; it does not prove who produced the event.

4. Proposed Architecture



Original bytes stay inspectable; normalized fields retain links to the source event.

Figure 1. Current prototype data path and review tools.

Core components

Component	Responsibility	Primary storage	Failure behavior
Browser UI	Ingest, browse, compare, approve; Next.js tabs and controls.	Browser state; API responses.	Shows disconnected/error state; retry after API recovery.
FastAPI API	Limits, routing, pagination, batch validation.	In-memory request data.	Rejects unknown source/invalid item; no queue retry.
Parser pipeline	Persist raw bytes, digest, parse, map, assess quality.	Raw files + SQLite metadata.	Parse/missing-field issues quarantine; storage crash can split writes.
Review tools	Evidence, drift, quarantine, replay, registry.	SQLite event/receipt/parser records.	No background recovery or multi-node coordination.
Simulator + dataset	Ten synthetic source families; optional bounded HF sample.	Demo events and import state.	WitFoo fetch needs internet; hosted demo may reset.

5. Request Lifecycle

1. Navigation. The nine tabs open Overview, Why LogProof, Live ingestion, Evidence explorer, Drift watch, Quarantine, Replay lab, Parser registry, and Pricing. The mobile selector opens the same views. Header status reports local, hosted, or disconnected API state.
2. Overview. "Start guided demo" seeds baseline examples and opens ingestion; "Explore evidence" opens the evidence list. "Trigger change" emits a firewall shape change and opens Drift Watch. "View evidence" opens the selected receipt. "Run replay" starts the parser comparison. Per-source controls set 1–5 events/second, emit one event, or pause/resume that source. The demo has no sign-in or role checks.
3. Live ingestion. "Start stream" / "Pause stream" control all simulator sources. "Emit firewall event" creates a valid sample; "Emit malformed event" sends broken JSON for quarantine. Select a source, edit raw text, then "Process raw event" submits it. "Import 20 WitFoo events" fetches a small sanitized sample and skips unavailable or repeated dataset rows. NDJSON "Import batch" accepts one source/raw or source/raw_base64 envelope per line; invalid lines are reported individually. "Export all NDJSON" downloads retained event objects. The ten demo families cover Palo Alto firewall JSON, Cisco-style Syslog, RFC 5424 Syslog, CEF, NGINX combined access, Windows security JSON, Windows Event XML, network-flow CSV, LEEF, and application JSON. Recent receipts refresh automatically; "Inspect" opens evidence and Previous/Next changes the server-paged list (20 items).
4. Evidence and quality. Search filters receipts or known sources and returns 20 items per page; selecting a row shows raw bytes, normalized fields, receipt/parser IDs, hash status, and field map. Selecting a mapped field highlights its source byte range when available. Drift Watch compares event shape with the baseline; "Trigger format change" emits the demonstration case and "Inspect" opens its receipt. Quarantine lists held events and reasons; "Inspect" opens retained evidence. These lists do not offer a quarantine reprocess action.
5. Replay and parser registry. Replay runs golden firewall cases plus up to 20 recent distinct firewall events against the same corpus. "Run replay" reports coverage, regression, timing, and field differences. "Approve & promote"

is enabled only after gates pass and an approver name is entered; it records the approval and previous version as rollback target. Registry "Rollback" restores that target. This managed promotion path is implemented for the firewall example only.

6. Purpose and pricing. "Follow an evidence trail" / "Inspect evidence" / "Trace a field" open Evidence; "See the parser gate" / "Open replay lab" open Replay; "See drift watch" opens Drift. Pricing tabs switch between annual licenses and a proposed pilot; "View pilot scope" selects the pilot tab. Displayed proposal: Community ₹0; Team ₹24,000/month billed annually; Secure ₹1.25 lakh/month billed annually; Enterprise/Government from ₹30 lakh/year; 12-week pilot ₹4.5–7.5 lakh one-time. Prices and listed enterprise features are illustrative, not a live offer or a statement that every feature is implemented.
7. Reset. Footer "Reset demo" stops streaming, removes demo events and raw receipt files, resets parser state, reseeds baseline events, and returns to Overview. Treat it as destructive to the current demo dataset. Failures appear in the relevant screen; retry after correcting source, payload, or connectivity. The WitFoo sample import requires internet access.

6. API and Data Contracts

Primary data contract

Field	Type	Required	Description
event_id	string	Yes	Normalized event identifier.
receipt_id	string	Yes	Links event to raw receipt and evidence.
source_id / format	string	Yes	Selects one of ten demo source-family parsers.
event_time	string	Usually	Parsed source time where available; missing required fields can quarantine.
raw_sha256	hex string	Yes	Digest for stored-byte integrity verification.
parser_version	string	Yes	Parser build used to produce the record.
field_map	array	No	Source key, transform, confidence, optional byte offsets; normalized fields can include severity, category, action, source/destination IP and port, user, rule, protocol, provider, channel, and facility. Not every source supplies every field.

Contract guarantees

- a. Raw bytes are written before parsing. A receipt and SHA-256 digest connect the stored object to the resulting metadata. Parser failure or missing required fields holds the event in quarantine; it does not discard the raw payload.
- b. Each accepted submission receives a new event and receipt ID. Listing order is newest first with limit/offset pagination; no exactly-once guarantee is claimed. Dataset imports track source row identity to avoid importing the same sample row again.

- c. Evidence records retain source, format, parser version, raw-file reference, digest, quality result, and field-map metadata. Replay uses a shared corpus for baseline and candidate. The local parser pack checksum detects file changes; packs are not digitally signed.
- d. The digest supports byte-integrity checks within this service. It is not proof of source-device authenticity, trusted time, or chain of custody. Hosted demo storage is shared synthetic state and may reset after redeploy.

Prototype API contracts are implemented in backend/main.py. Routes include POST /api/ingest/{source_id}, POST /api/ingest/batch, GET /api/events/page, GET /api/events/{event_id}, GET /api/evidence/{receipt_id}, GET /api/events/export.ndjson, GET /api/drift, GET /api/quarantine, POST /api/replay, GET /api/parser/registry, POST /api/parser/approve, POST /api/parser/rollback, and /api/simulator/*. Standards references: RFC 5424 <https://www.rfc-editor.org/rfc/rfc5424> · NDJSON <https://github.com/ndjson/ndjson-spec> · WitFoo dataset <https://huggingface.co/datasets/witfoo/precinct6-cybersecurity-100m>

7. Consistency, Idempotency, and Replay

This is a synchronous, single-service prototype. Filesystem writes and SQLite metadata are separate operations, so a crash can leave an orphaned raw file or incomplete receipt; no transaction spans both stores. Repeated manual uploads create new receipts. Batch items are processed independently. Simulator and dataset imports are bounded; there is no queue, distributed ordering, or exactly-once delivery.

Scenario	Observed behavior	Reasoning / limit
Repeated manual upload	New receipt/event ID. Dataset row duplicates are skipped.	No exactly-once guarantee.
Bad syntax / missing required fields	Raw event retained; parse result quarantined.	Preserves inspectable evidence.
Batch item invalid	That line is rejected; other items continue.	Per-item partial success.
Parser change	Same firewall corpus replayed; promotion gated on passing checks.	Narrow demo workflow, not general parser lifecycle.

8. Security and Privacy Considerations

- a. The demo has no authentication, authorization, or tenant isolation. Run local-only with trusted operators. Do not expose it to untrusted networks or upload sensitive production logs.
- b. Raw input is retained for evidence, so payloads may contain secrets or personal data. Use synthetic/demo data, restrict filesystem access, and define a retention/deletion policy before real logs.
- c. Dataset fetching uses public Hugging Face access and may require outbound internet; no credential vault or key-management workflow is implemented. Configure deployment secrets outside source control.
- d. Reset deletes current demo events and raw receipt files. Promotion is gated by replay results and an approver string, but the string is not authenticated. Review changes before using this prototype with anything other than demo data.
- e. Local mode stores data under the configured data directory; Docker can persist it on a mounted volume. Hosted demo data is shared and may reset. Per-tenant retention, secure erase, residency, backups, and audit exports remain to be designed.

9. Operational Readiness

Signal	Current evidence	Owner	Production gate
Throughput	1,000 event sequential local benchmark; illustrative only.	Not assigned	Define workload + repeatable target.
Latency	Replay reports elapsed time; no service SLO.	Not assigned	Measure p95/p99 under concurrency.
Errors	API responses and quarantine reasons.	Not assigned	Add durable alerts and on-call owner.
Configuration	Parser version/checksum shown in registry.	Not assigned	Add signed pack and controlled distribution.
Integrity	SHA-256 verification shown in evidence view.	Not assigned	Test backup, recovery, and custody requirements.

10. Alternatives Considered

Alternative	Why considered	Current reason not selected
Direct SIEM ingestion	Fewer components.	Leaves parser provenance/replay assurance outside this demo.
Existing pipeline only	Mature transforms and connectors.	LogProof prototype demonstrates an upstream review layer; no parity claim.
Cloud managed service	Fast to operate.	Air-gapped target and raw-data control need local evaluation.
Full event platform	Broad analytics.	Out of current prototype scope; build assurance proof first.

11. Open Questions

1. Which SIEM, data lake, and export contract must be supported first?
2. What authentication, roles, retention, and data residency rules apply to the ministry deployment?
3. What measured peak rate, burst size, and availability target should guide queueing and storage design?
4. Who owns parser approval, signed releases, offline distribution, and incident response?

12. Decision and Next Steps

Treat LogProof as a prototype for validating evidence traceability and safe parser changes. The next step is a restricted pilot plan: select representative sanitized sources, define access and retention, add authenticated roles and durable transactional storage, test export into the target SIEM, then run a separately measured offline deployment and load test. Broader ministry use should wait for threat modeling, operational ownership, recovery tests, and agreed scale targets.

Milestone	Deliverable	Exit criteria
M1	Threat model, target sources, data policy.	Owners and scale/security requirements approved.
M2	Auth, transactional persistence, signed parser pack.	Security and recovery tests pass.
M3	SIEM export and offline transfer on separate host.	End-to-end trace plus no-network install verified.
M4	Measured pilot under agreed workload.	Load, failure, rollback, and operator acceptance gates met.